

思维导图软件课程个人报告

报告人：朱拓源

一、项目描述

1.1 项目概述

本项目是一个基于Python和PyQt5开发的思维导图编辑软件，该软件在完成完整的思维导图创建、编辑、保存和管理功能的同时，额外开发手写批注、界面定制等扩展功能，以连线管理为软件特色，在学习、工作场景下提供制作思维导图的便利。

1.2 核心功能

- 节点管理**：具有创建、删除、修改各级主题节点的基础功能，支持创建中心主题、分支主题、子主题和自由主题，节点文本支持富文本编辑，可输入不等长文字信息，节点颜色和文本颜色可自定义，节点移动支持移动子树模式。
- 连接线管理**：界面自动生成父子节点之间的连接线，支持手动管理连接线（创建或删除），连接线自动跟随节点位置进行调整。
- 文件操作**：可新建、打开、保存思维导图文件（XML格式），具有自动保存功能（每10秒间隔保存一次），支持导出为PNG图片格式、PDF文档格式，有打印功能，同时可以从Markdown文件导入为思维导图。
- 编辑功能**：各操作可进行撤销（Undo）和重做（Redo），可复制、剪切、粘贴节点，删除节点支持删除子树）。

1.3 扩展功能

- 节点附加信息**：可为节点添加备注信息、超链接、文本批注、插入图标、待办事项，丰富节点内容。
- 手写批注**：支持手写绘制批注，可切换笔模式和橡皮擦模式，自定义画笔颜色和粗细，通过框选删除批注。
- 界面定制**：提供三种外观主题：浅色（可爱风格）、深色、黑白，具有中文/英文切换的双语支持，支持思维导图放大和缩小缩放视图，状态栏显示当前节点数和操作提示。
- 用户体验**：该软件有友好的图形操作界面，左键虚线框选多节点，右键菜单快捷操作，工具栏呈现快捷按钮。

二、系统描述

2.1 系统架构

本系统采用MVC架构模式实现模块解耦与功能扩展，通过Qt信号/槽机制实现层间通信，设计框架如图2-1所示。架构分为三大层次：

- Model层（图元层）**：

Graph类（场景管理）：继承QGraphicsScene实现场景管理，支持节点/连接线的增删改查、XML数据持久化、PNG/PDF导出、Markdown导入及手写批注管理。采用树形数据结构存储节点关系，确保子树移动时自动维护连接线拓扑。

Node类（节点绘制）：基于QGraphicsTextItem封装节点绘制逻辑，实现圆角矩形背景渲染、富文本编辑、双色自定义、图片/链接/待办事项插入及父子关系管理。支持不等长文本自动换行与焦点事件响应。

Branch类（节点连接）：继承QGraphicsLineItem实现动态连接线，通过信号监听节点位置变化自动调整线端坐标，支持手动创建/删除连接线及线型样式配置。

Annotation类（手写）：包含AnnotationItem绘制项与HandwritingDialog设置对话框，实现手写批注的笔模式/橡皮擦模式切换、画笔参数自定义及框选删除功能。

• **View层（UI层）：**

Mainwindow类（主界面）：集成菜单栏、工具栏、状态栏及CustomGraphicsView自定义视图。实现文件操作（新建/打开/保存/自动保存）、编辑功能（撤销/重做/复制/粘贴）、主题切换（浅色/深色/黑白）及双语支持。

Component类（其他组件模块）：封装UI组件模块，包括备注编辑窗口、链接编辑窗口、批注编辑窗口、节点信息窗口、待办事项列表对话框及自定义缩放滑块。采用模块化设计支持功能扩展。

• **Controller层（命令层）：**

Command类（撤销重做）：基于QUndoCommand实现命令模式，封装节点插入/删除、移动、颜色修改等操作。通过撤销/重做栈实现操作历史管理，支持无限次回溯与事务性操作。

系统通过Config类进行全局配置，定义节点层级常量、窗口尺寸参数及自动保存时间间隔（默认10秒），支持主题配置与语言切换的持久化存储；以Main作为程序入口，初始化Qt应用、创建主窗口、建立信号/槽连接并启动事件循环，采用单例模式管理全局配置与资源加载。

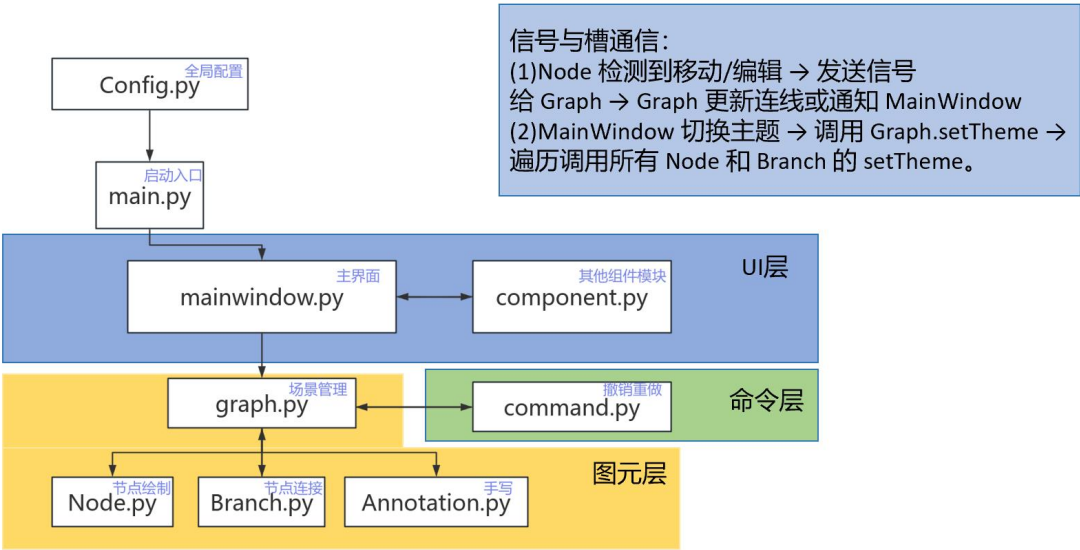


图2-1 设计框架示意图

2.2 核心模块

2.2.1 main.py - 程序入口

初始化Qt应用程序，设置应用属性；创建主窗口实例，加载全局配置；建立窗口间信号/槽连接，实现菜单操作、工具栏按钮与后台逻辑的联动；启动应用程序事件循环，监控系统事件。

2.2.2 mainwindow.py - 主窗口

MainWindow管理主窗口布局，集成菜单栏（文件/编辑/视图/帮助）、工具栏（新建/保存/剪切/粘贴等快捷按钮）、状态栏（节点数/操作提示）及CustomGraphicsView视图。

CustomGraphicsView继承QGraphicsView，扩展虚线框选功能（支持多节点选择）、手写模式切换及视图缩放控制。通过滚轮事件实现无极缩放，配合缩放滑块同步显示缩放比例。

2.2.3 Graph.py - 场景管理

实现XML文件解析与生成，采用DOM模型存储节点层级关系、文本内容、样式属性及连接线信息；支持PNG/PDF导出时自动计算场景边界，适配不同分辨率与页面尺寸；Markdown导入功能解析标题层级、列表项及链接，自动转换为思维导图节点结构；手写批注管理模块支持批注图层的叠加显示、编辑锁控制及数据同步。

2.2.4 Node.py - 节点类

节点绘制采用抗锯齿渲染，背景色与文本色支持RGB/十六进制值自定义；富文本编辑集成Qt富文本引擎，支持字体样式、段落对齐及列表项渲染；待办事项列表实现复选框交互与状态同步，支持子任务嵌套与进度统计。

2.2.5 Branch.py - 连接线类

连接线自动避障算法优化路径规划，避免节点重叠时线条交叉；支持连接线样式配置（实线/虚线、箭头类型、线宽），并提供手动调整端点位置的交互手柄。

2.2.6 Command.py - 命令模式

命令栈实现操作原子化，每个命令记录操作前后的状态差异；支持批量操作的事务性提交，确保撤销/重做时数据一致性。

2.2.7 Component.py - UI组件

待办事项列表对话框支持任务排序、截止日期设置及完成状态标记；自定义缩放滑块集成滑块事件与视图缩放比例的双向绑定，实现无闪烁实时渲染。

2.2.8 Annotation.py - 手写批注

手写批注采用矢量路径存储，支持笔画平滑处理与压力感应（若设备支持）；批注编辑对话框提供画笔颜色/粗细调节、橡皮擦模式及撤销/重做功能。

2.2.9 Config.py - 配置常量

节点层级定义支持四级结构（中心主题/分支主题/子主题/自由主题），对应不同默认样式与交互行为；窗口尺寸配置适配主流显示器分辨率，支持响应式布局调整。

本架构通过模块化设计与Qt信号/槽机制实现高内聚低耦合，支持功能扩展与跨平台部署，为思维导图创作提供高效、稳定的技术底座。

三、个人开发经验

在本次软件课设项目开发的过程中，我承担了扩展功能实现、BUG测试等多个任务，是完成该项目的代码手。在学习pyqt5以及掌管代码书写时学习到了很多软件开发的知识和经验，现做出汇报如下：

1. 我在项目中完成的工作：在Node.py中添加node类的坐标属性刷新和鼠标拖拽信号的传递方法，实现节点拖拽移动功能；编写Annotation.py文件，调用pyqt5中画笔的对象，实现了鼠标拖拽时虚线框的绘制；修改选中节点的逻辑，实现更加人性化的为节点连线的功能；在扩展部分，实现todolist待办事项的功能；后期BUG调试；项目稳定性测试等等。

2. 开发工具的使用：在本次代码中，我们小组使用了 git 进行项目管理。在完成整个项目的过程中，我非常开心自己在学习了项目代码之余掌握了多个 git 的指令集，这有助于我们更好地管理项目版本。在实现拖拽节点的这个任务中，我不小心修改了代码的核心结构导致程序运行失常，幸好我使用 git 记录了代码，使用指令回退到原先的版本，git 对我来说是一个不可或缺的工具，这次开发经历也提醒了我需要在书写代码的同时掌握更多的工具集，以便在软件设计时提高自身工作效率。

3. 环境搭建的过程：本次团队项目开发中我第一次尝试以编写博客的方式快速和团队其他成员一起熟悉 python 环境的搭建。博客源网址：https://blog.csdn.net/2302_79819003/article/details/152336209。在综合考虑性能、复杂度、任务需求的情况下，我们最后选择了anaconda完成项目的python环境开发。使用anaconda的好处在于隔离多种版本冲突，自动解决版本依赖。python是一门前景广阔、可实现多种任务且易上手的语言，在以后使用anaconda的过程中，我要更加注意对版本的管理，同时探索其他轻量便携的环境搭建方案，例如UV。同时，本次项目对于pyqt5库的使用也极大地激发了我学习python其他库的兴趣。我希望在以后的日子里，我可以更加熟练对于其他库的使用。

4. AI工具的思考：本次开发我也第一次尝试使用AI插件完成代码的书写，引发了我对于AI和开发者之间的深度思考。在项目开发过程中，我使用AI极大地增加了我书写代码的效率。但我认为效率的前提在于开发者需要熟练地掌握代码的实现原理。开发者不应该对于自身使用的语言‘不熟悉’‘不理解’‘看不懂’，从而‘两眼一抹黑’，把提示词喂给AI之后就当‘甩手掌柜’，等着AI吐出来结果。但其实这样的开发无法使开发者全面掌握代码运行的动态过程，从而也无法完成代码的修改和调试，最后的结果也只会谬以千里。一个好的开发者，在使用AI的同时，我认为应该做到如下几点：1. 微观上理解类和方法的具体实现过程，具备基本解释代码的能力 2. 宏观上掌握代码运行流程，代码运行机制 3. 平时需要积极学习各类知识，从而当任务来临时，具备多途径解决该问题的能力。我认为开发者仍需要具备不断学习完善自己的能力，人类彻底解放写代码的双手而交给AI的未来，我认为还有很长一段距离。

5. Pyqt5的学习：在本次学习python库的过程中，我认为我有以下的宝贵经历：首先，我学习的是该库的官方文档：<https://doc.qt.io/archives/qtforpython-5/contents.html> 尽管文档全部都是英文，但是我从pyqt5官方文档中可以学习到许多对于API的调用，了解许多组件被设计出来的思想。我非常享受从官方文档中学习代码的过程，因为这教会我一个技术的应用需要依赖规范化的设计。我希望以后自己可以提升阅读文档的能力，从官网中学习更多的知识，提高并深化自己的技术水平。

6. 在开发过程中面临的问题和挑战：在书写代码并进行调试的过程中我遇到了一些阻力和困难，但是这是非常正常的情况，所以我需要做的就是摆正自己的心态，积累自己解决问题的过程和经验。首先，一开始对node节点试图选中时，我发现自己没有办法成功同时选中多个节点，这会导致该软件不利于使用者操作的习惯。在经过了大量代码的阅读和调试之后，我意识到自己可以通过建立一个栈保存选中属性为真的所有节点，并将这些属性交给mainwindow进行管理即可。所以经过了辛苦的调试，我最后也解决了这个问题。这次经历让我意识到代码在修改之后需要积极参与调试，同时分析每个对象的属性，在有了全面的认知之后解决问题就会更加得心应手。

7. 丰富了自己团队协作的经验。在以前我非常热衷于独立开发，认为只有我自己才能管理自己的代码，如果团队的所有人共同管理一份代码，那么整个任务将会变得不协调、很难统一。抱着这样的想法，在过去的时间内我一直都是一个‘独狼’。但是这次团队协作在整体程度上远大于我个人独自开发的效率，我认为首先我需要特别感谢我的队友，他们会提醒我，如果我修改了代码就要在群聊里面说一声让大家知道，他们负责任的态度让我改正了过去独自开发的习惯。所以在代码功能实现的过程中，我和我的队友会交流讨论，将目前调试的结果以视频的形式发到群聊里，让队友实际感受到具体功能的实现。同时，我认为我的队长使用markdown书写开发日志是一个很好的习惯，我认为值得借鉴。同时在团队协作的过程中我认为还要善于书写注释，养成写注释的习惯非常有利于团队协作。

8. 创新的产品设计理念。我认为本次开发过程中，我还有一个可取之处就是精益求精的态度。我并没有把这个课设软件当作一个应付交差的任务，而是把他当作一个可以被更多人使用的一个产品。写代码的

过程中，我也在试着为自己解答这样的问题：如何让人们意识到思维导图是一个可以被使用、非常便携的工具呢？基于此问题，我给这款软件添加了以下功能：一、todolist待办事项。制作思维导图本身是一个长期的过程，学习也不是一蹴而就的，因此思维导图会随着一个人学习过程中搭建的思维框架而有所变化，所以我认为使用这个软件的人需要这个软件有一个todolist功能，用来记住需要补充而暂时没能放进思维导图内部的idea。二、一键管理连线。选中需要的节点并点击连线管理按钮，那么未被连线的节点会出现连线。这里面的一个底层想法就是，用户不管做出了什么操作，在画面上都需要都及时反馈的结果，哪怕这个反馈不符合用户的心里预期（即连线有重复或遗漏），用户也可以继续调整自己的操作直到得到满意的结果。在写代码的过程中，这些思考的过程也将成为我的一部分财富，帮助我在以后的产品设计上找到更多的灵感。

9. 反思和不足。首先，该项目如果删除节点，有时候伴随的连接线会卡住而删不掉，这是我们在软件测试过程中没有留意的一个地方。或许下次我可以列一张测试表，一项一项进行排查。其次，我认为我的UI设计不算特别好看，我希望以后这个项目可以进行优化，在背景板上添加一些诸如方格纸、草稿纸的背景，这样用户在做思维导图的同时还可以在旁边记笔记；在图标设计上我希望以后可以在背景上选择是否添加水印，比如华中科技大学的logo，这样这个思维导图软件可以被华科的老师或学生用于制作教学课件或汇报总结的课件。诸如此类的需求还有很多，我希望以后有时间我可以将他们补充上去。总之，一款好的软件设计离不开日复一日的打磨、开发者的精益求精的态度。在未来的软件设计道路上，我希望自己依然能够静下心来，完善丰富作品的功能，让作品更加适应多元化的场景，做出让自己满意、让使用者满意的作品出来！